

The Reactor

The convenience of a centralized SaaS with the sovereignty of local-first.

REQUIREMENT 1

Local-first

Data *and* execution live on the user's machine. Every peer is authoritative. Work continues offline.

RULES OUT

- ✗ A privileged server copy that "wins"
- ✗ Reducers that only run server-side

FORCES

- Command sourcing: ship intent, replay it locally
- One runtime for browser, server, CLI, mobile
- Eventual consistency between peers

REQUIREMENT 2

Append-only

The operation log is a hash-chained audit trail on append-only backends (Swarm, Hypercore). History is never edited.

RULES OUT

- ✗ Reordering or rewriting past operations
- ✗ Deleting operations that turn out stale

FORCES

- Create new operations instead of editing old ones
- Skip values + a garbage-collected view
- Every projection is rebuildable from the log

REQUIREMENT 3

Signed intent

Every Action is signed by its author and must verify on every peer. Nobody's mutation may be altered by someone else.

RULES OUT

- ✗ Transforming other users' operations (OT)
- ✗ Silently discarding a peer's work

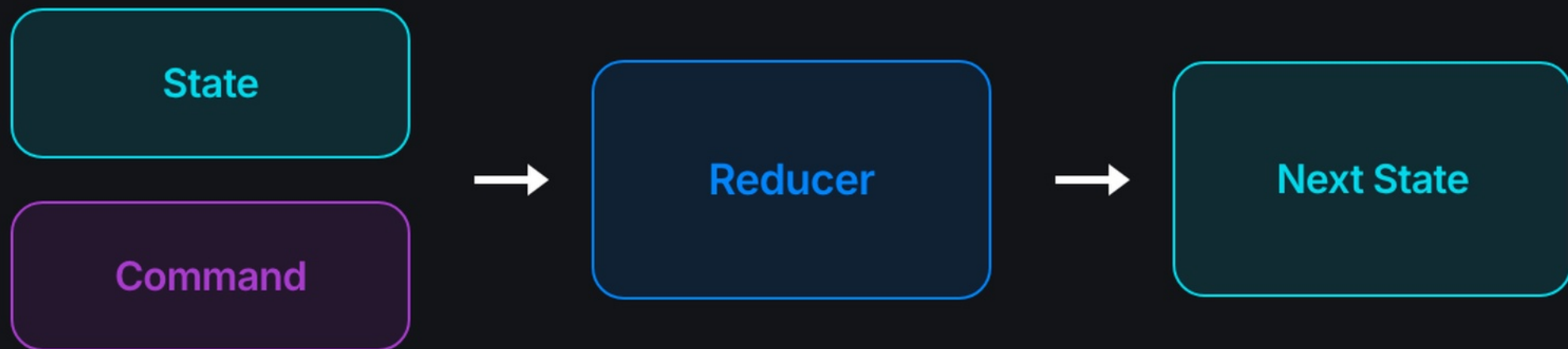
FORCES

- Replay Actions verbatim; new Operations only
- Signature tiers that can bind state and order
- Dead-letter anything that fails verification

AGENCY AND OWNERSHIP FOR THE USER · THE CONVENIENCE OF CENTRALIZED SAAS · NO MAGIC

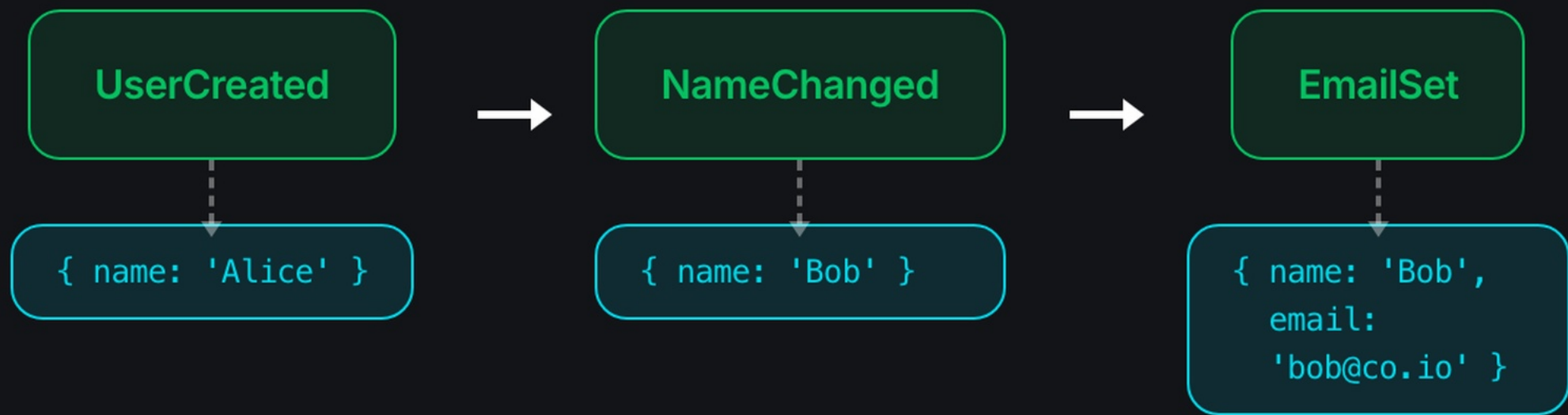
Reducer Update Loop

Unidirectional state transitions via pure functions



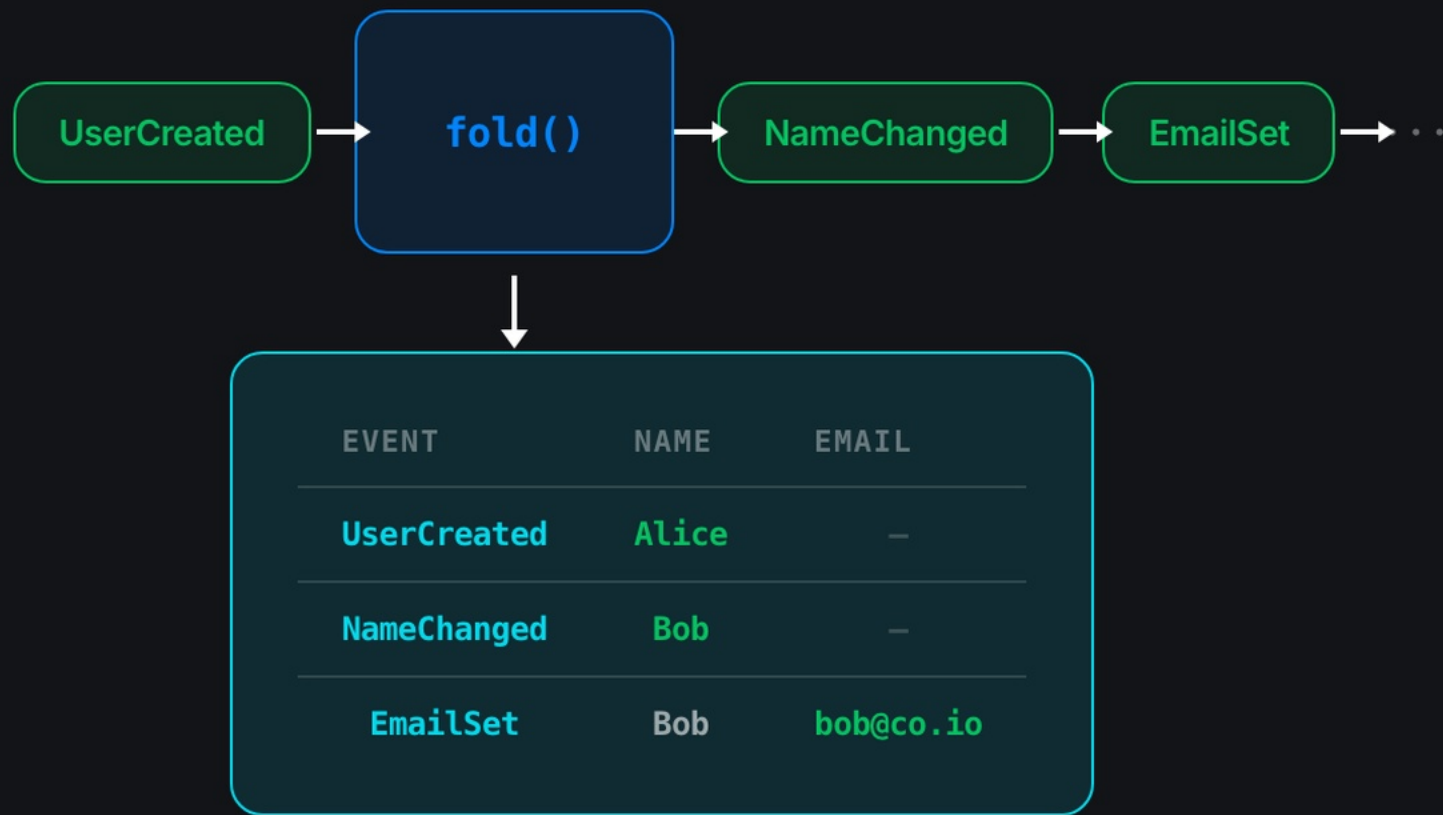
Event Sourcing

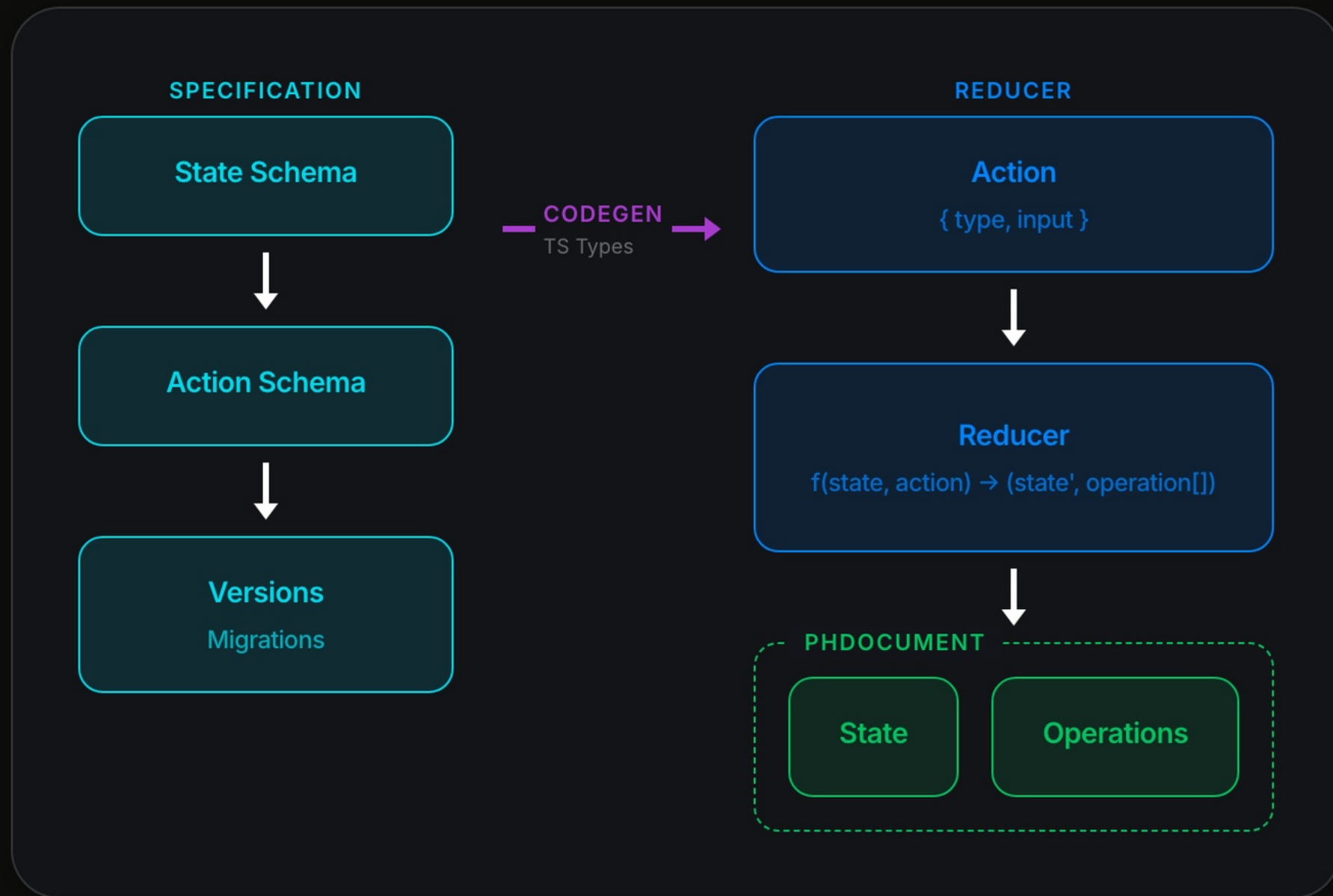
State as a stream of events



Aggregate

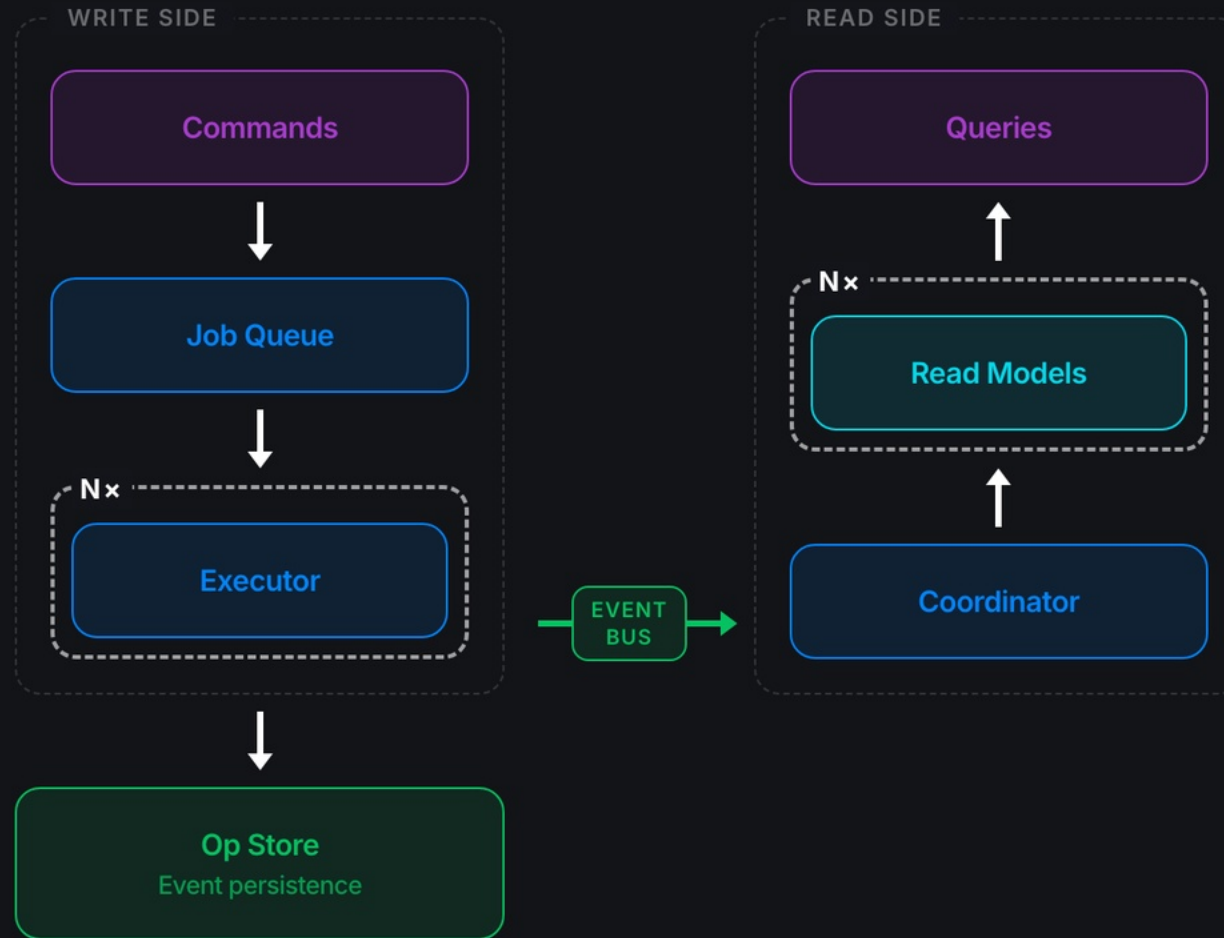
Folding a stream of events into a result

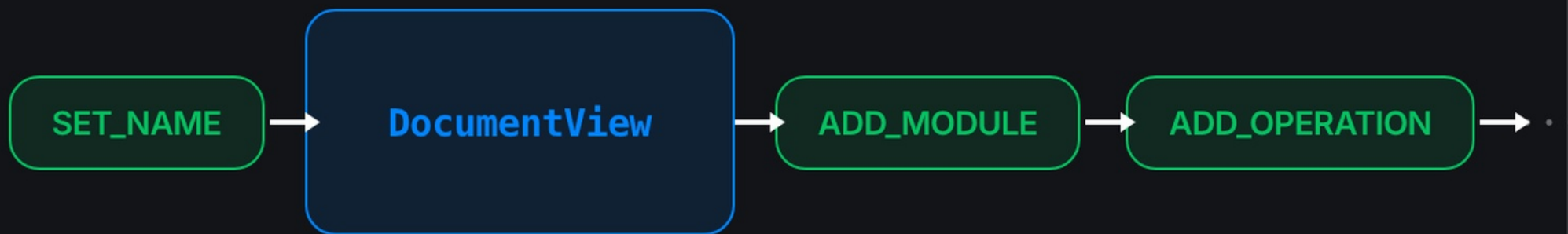




Reactor CQRS

Write/read side separation with independent scaling



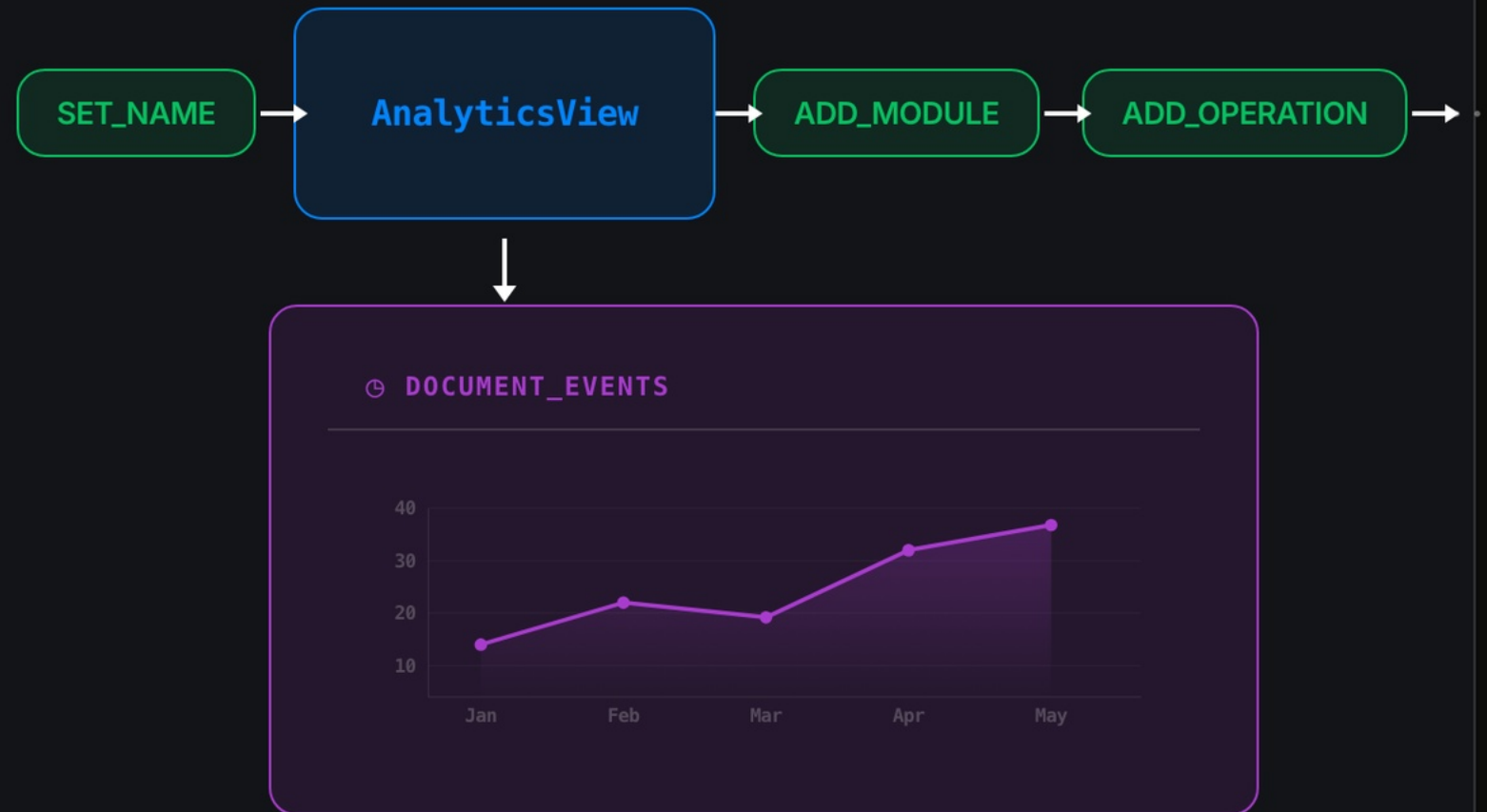


▼ DOCUMENTS

ID	STATE	MODEL	UPDATED_AT
doc_a1	{ "name": "Budget Q1", ... }	budget/v1	2025-01-15
doc_b2	{ "total": 4200, ... }	invoice/v2	2025-02-03

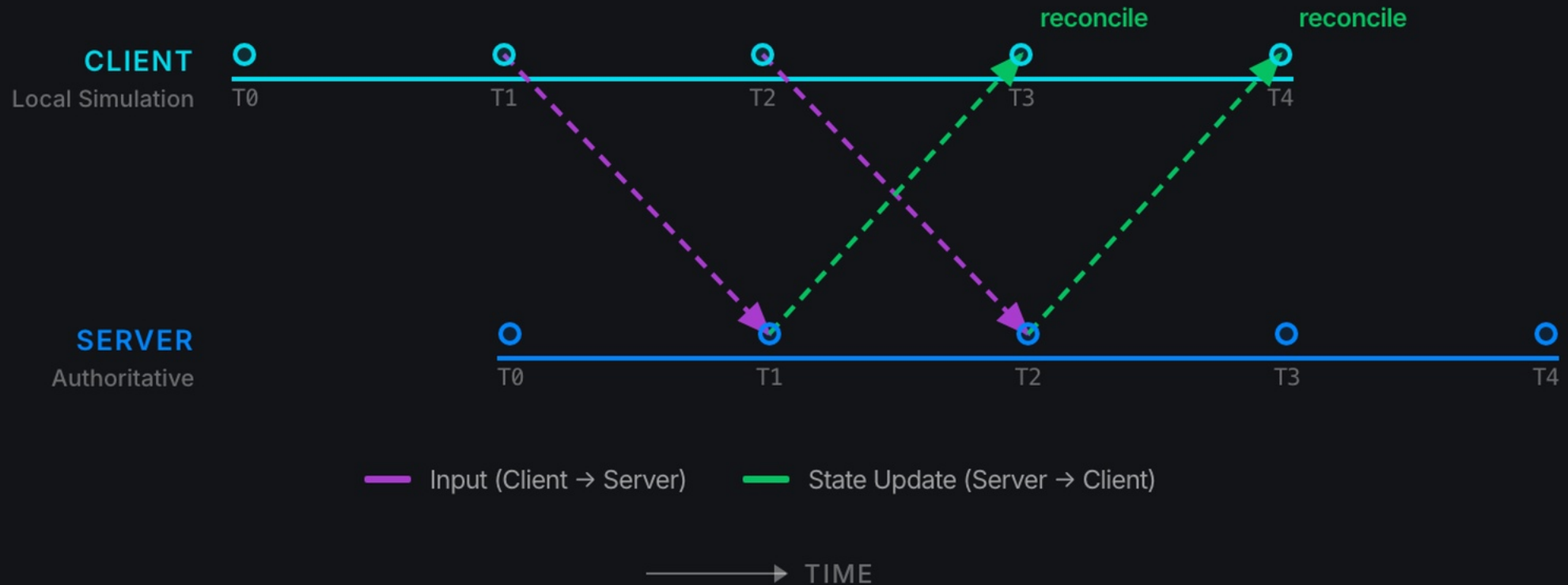
Analytics Read Model

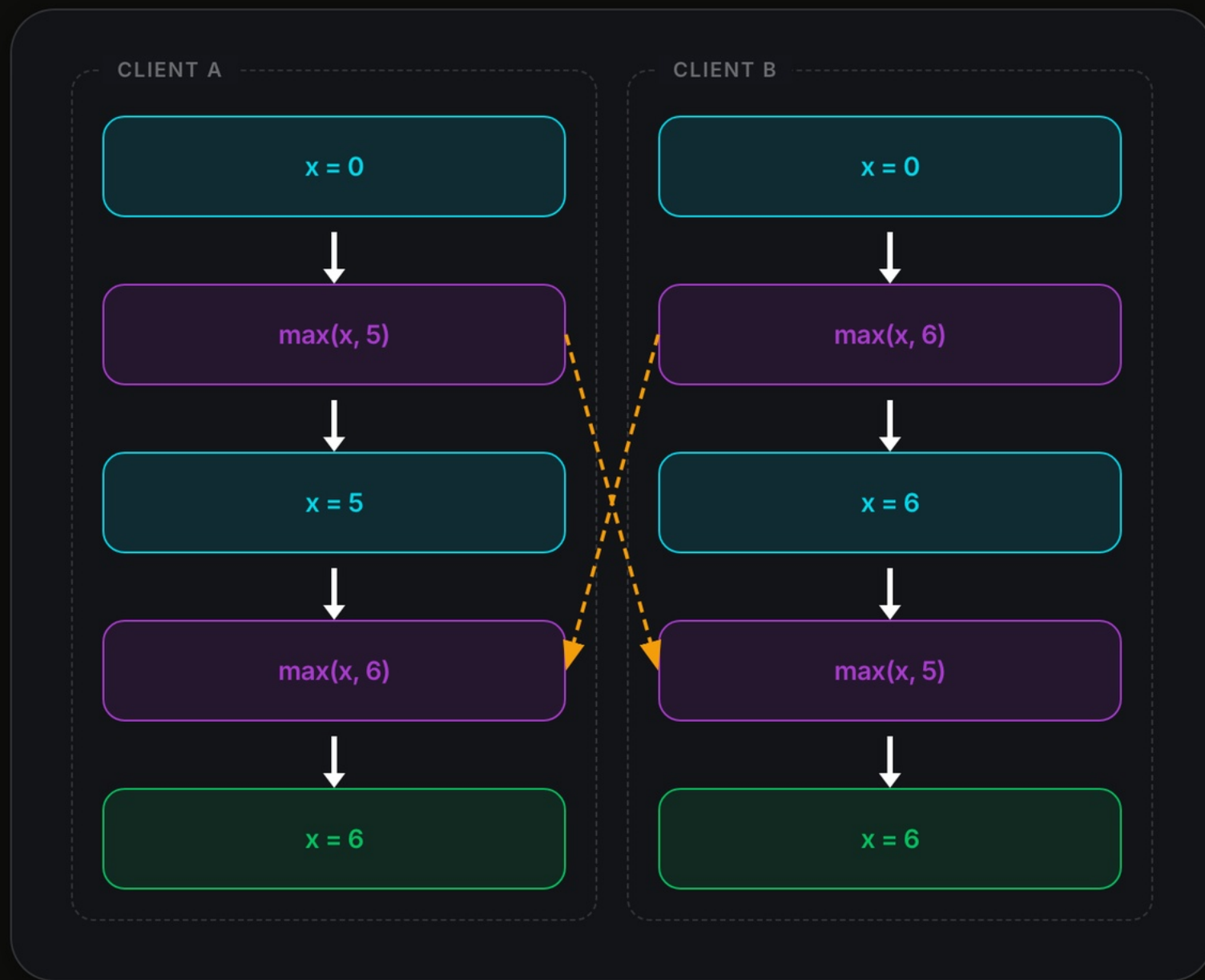
Collecting change analytics into a time-series database

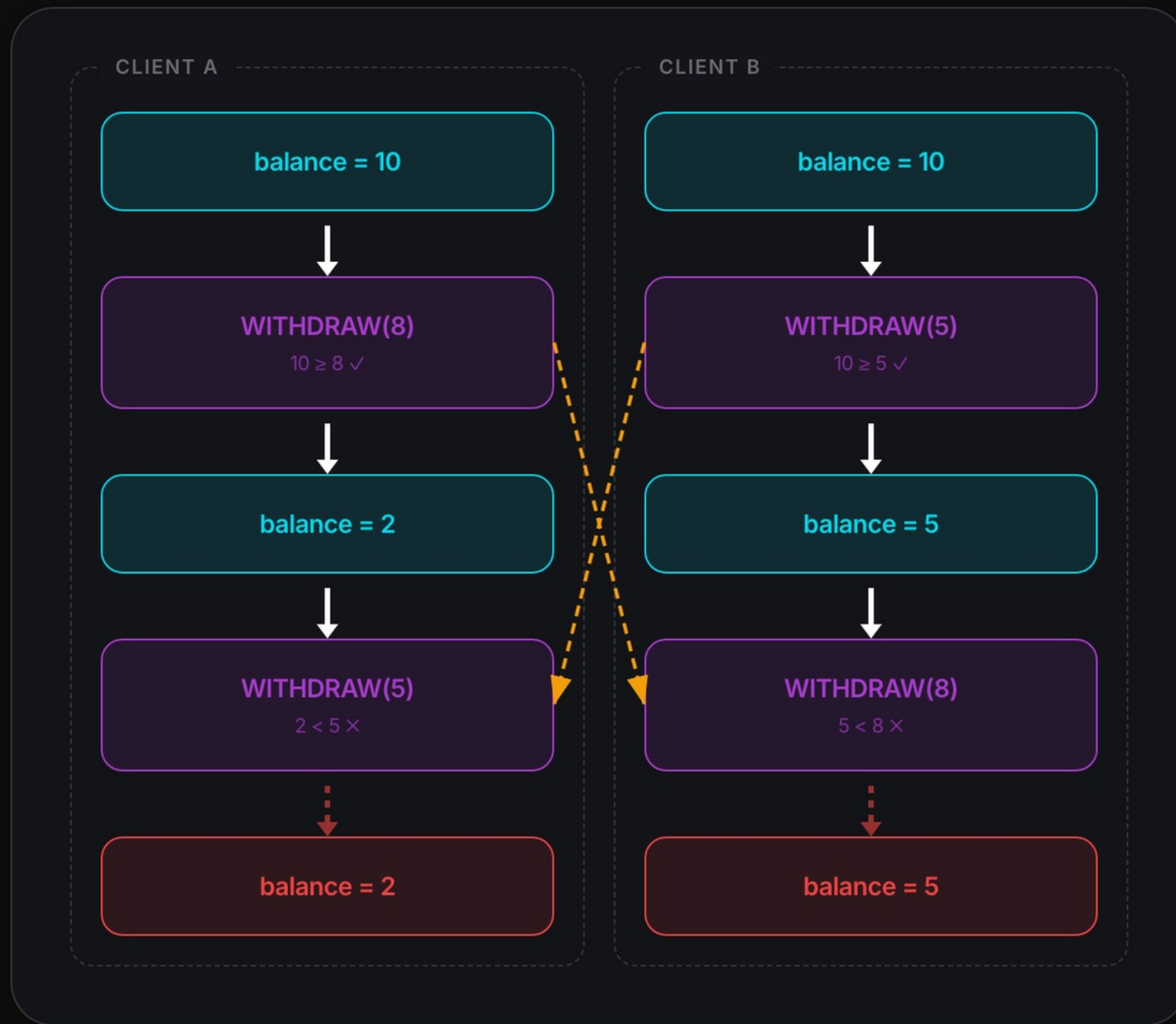


Client / Server Game Timelines

Local simulation runs ahead; authoritative simulation confirms







Interleaving Breaks State Hashes

Each operation records its previous state hash — interleaving invalidates the chain



Every operation after the first mismatch must be rewritten with new hashes.

	EVERY PEER AUTHORITATIVE	APPEND-ONLY LOG SURVIVES	INTENT NEVER REWRITTEN	ANY REDUCER (ORDER-DEPENDENT OK)
Authoritative server multiplayer games	 one privileged simulation	 not a concern	 server stomps client work	 any simulation logic
CRDTs Figma-style	 converge without a leader	 ops are applied as-is	 ops are applied as-is	 ops must commute; no balance checks
Operational Transformation Google Docs	 usually server-mediated	 rewrites past operations	 others alter your mutation	 transforms are hard to generalize
Operational Reshuffle The Reactor	 every Reactor reshuffles	 new ops + skip; nothing edited	 Actions replayed verbatim	 replay in order; state-bound sigs if needed

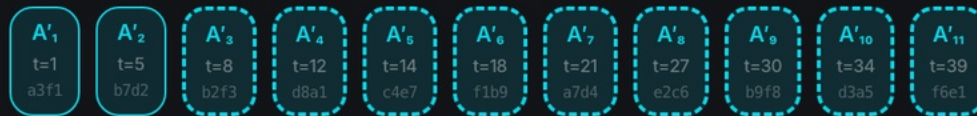
Reactor A

Reactor B

reshuffled ops

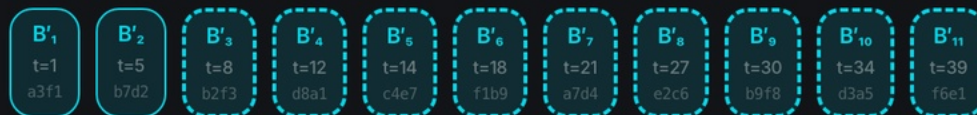
reshuffled ops

GARBAGE COLLECT



$A' - 11$ OPERATIONS

=



$B' - 11$ OPERATIONS

$A' = B' - \text{identical streams}$

Payload-only signature

DEFAULT

PREVIOUS OPERATION



NEXT OPERATION



✗ **Independent of state**
signature remains valid even when state changes due to reshuffle

Payload + input-state hash

HIGH SECURITY

PREVIOUS OPERATION

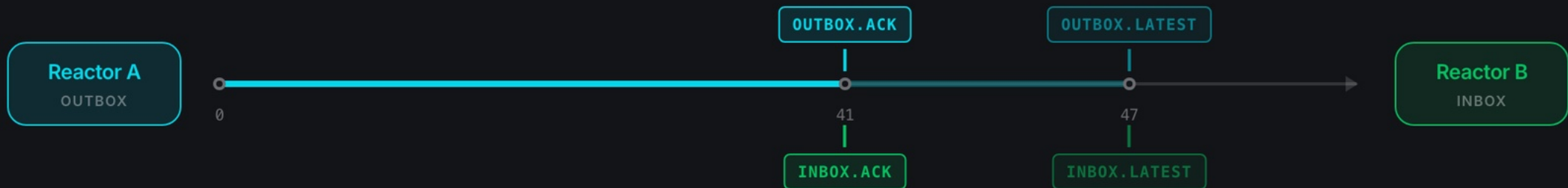


NEXT OPERATION

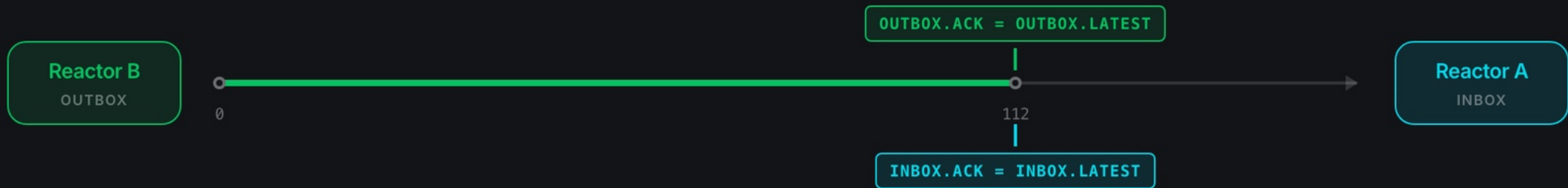


✓ **Bound to state**
action requires an explicit state

A → B · A'S OUTBOX / B'S INBOX · A'S ORDINALS



B → A · B'S OUTBOX / A'S INBOX · B'S ORDINALS



ack ≤ latest

cursors only move forward

synced ⇔ ack == latest

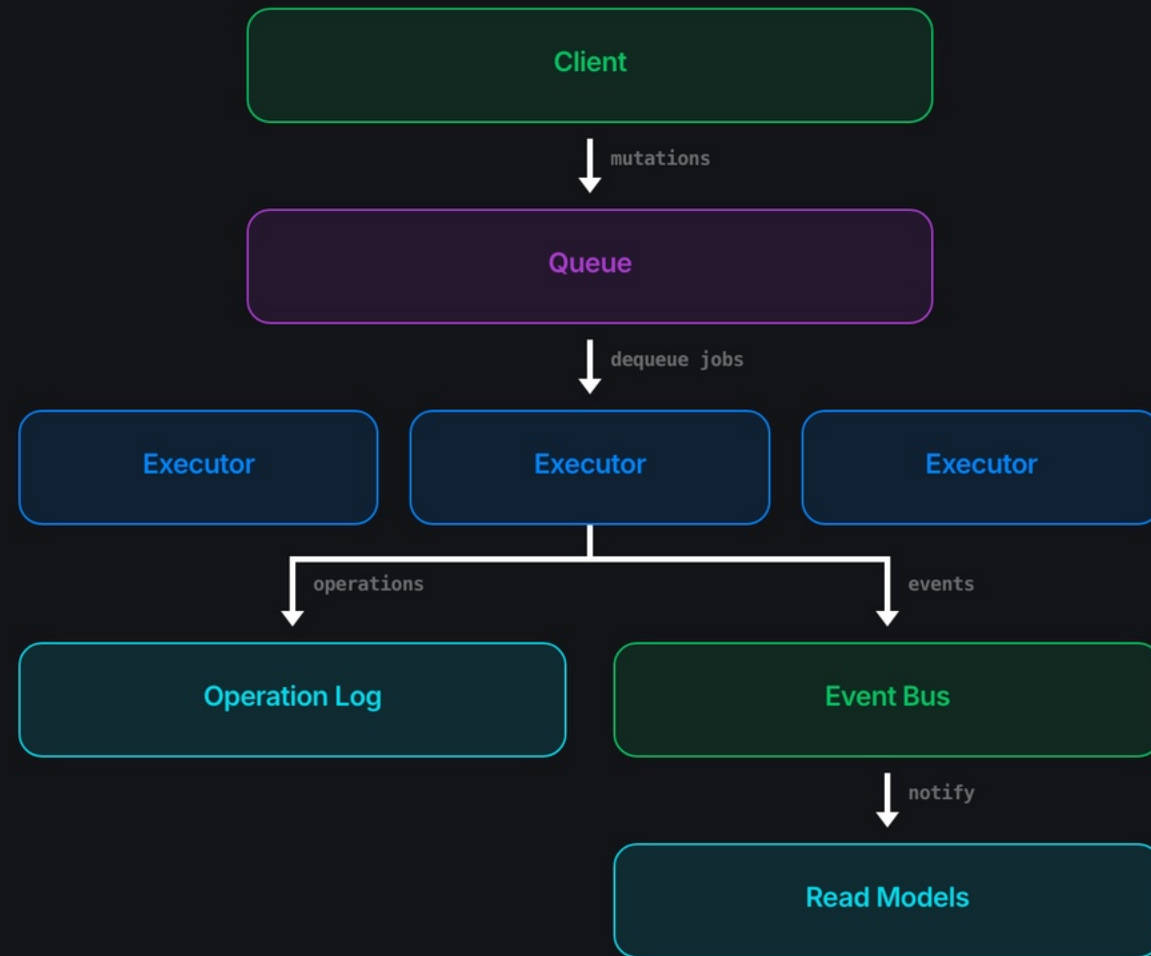
	EVERY PEER AUTHORITATIVE	APPEND-ONLY LOG SURVIVES	INTENT NEVER REWRITTEN	ANY REDUCER (ORDER-DEPENDENT OK)
Authoritative server multiplayer games	 one privileged simulation	 not a concern	 server stomps client work	 any simulation logic
CRDTs Figma-style	 converge without a leader	 ops are applied as-is	 ops are applied as-is	 ops must commute; no balance checks
Operational Transformation Google Docs	 usually server-mediated	 rewrites past operations	 others alter your mutation	 transforms are hard to generalize
Operational Reshuffle The Reactor	 every Reactor reshuffles	 new ops + skip; nothing edited	 Actions replayed verbatim	 replay in order; state-bound sigs if needed

Thanks

Questions?

Reactor Architecture

Job processing pipeline



Payload + input-state hash + id

CRITICAL SECURITY

PREVIOUS OPERATION

id

state

+

action

NEXT OPERATION

id

state

+

```
{  
  type: "SIGN_STATEMENT",  
  payload: { ... },  
  signature: sig(payload + H(state) + id)  
}
```



Bound to state and order

action requires specific state and order

